CL-1 Endsem Answer key

Section-A

Question-1

a. **Explain how the morphological analysis influences POS tagging accuracy, especially in morphologically rich languages. Why is it important to distinguish between inflectional and derivational morphology in this context [2.5]**

   Explanation of morphological analysis influencing POS tagging accuracy, MRLs → **1.5** (**1** → how to use morph analysers for increasing POS tagger performance, **0.5** → morphologically rich langauge case specific)

   Importance of distinguishing between Inflectional and derivational morphology in this context → **1** mark (no change in tag, but extra information in case of inflection; change in tagset in case of derivational)

   Inflectional Morpheme: creates new forms of the same word with the addition of grammatical properties; the basic meaning (and the category) of the word is the same. eg:. with nouns: book, books, books', ride, rides, rode, ridden, riding (no change in tag)

   Derivational Morpheme: creates a new word with a different meaning that may belong to a different or the same grammatical category. EG: re+write ('write again') verb, write + er = writer ('one who writes'), noun

b. **Discuss the limitations of POS tagging in handling multi-word expressions (MWEs) and idiomatic phrases with examples [2.5]**

   identifying the limitations of POS tagging in handling MWE, idiomatic phrases → 1.5

   Few challanges:

   1. Ambiguity, →"take off"
   2. Variability → ""split the beans"
   3. Non-composionality: "kick the bucket" ⇒ POS tagger will treat it literally

   examples → 0.5 + 0.5

Question-2

a. **Explain how chunking contributes to NER with examples. Can NER be considered a subtask of chunking? Justify your answer by discussing both overlaps and distinctions between the two tasks. [2.5]**

   contribution of chunking towards NER → 0.5 + 0.5 (example)

   chunking is shallow parsing, does not go deep parsing or full syntactic analysis

   chunking groups possible entities and aids NER.

   No, NER and chunking are different → 0.5

   Justification = overlaps (0.5) + distinctions (0.5)

   Overlaps:

   - Both involve identifying coherent segments in text.
   - Both operate over multi-token expressions.
   - Chunking can be a preprocessing step that facilitates NER.

   Distinctions:

   - **Goal:** Chunking is syntactic; NER is semantic.

- **Labels:** Chunking outputs phrase types (e.g., NP, VP), while NER outputs entity types (e.g., PERSON, ORGANIZATION).
- **Dependency:** Chunking can help NER but is not required; modern NER models (e.g., BERT-based) often skip chunking entirely.

b. **Discuss the relationship between MWEs and NER. Are all named entities MWEs? Are all MWEs named entities? Justify with theoretical reasoning. [2.5]**

Multiword Expressions (MWEs) are sequences of words that function as a single unit and whose meaning is not always deducible from the individual words. **Named Entities (NEs)** are specific names of people, places, organizations, etc. → 0.5

Not all named entities are MWEs. → 0.5

No, most MWEs are **not** named entities. → 0.5

Theoretical Justification: → 1

- MWEs cover a **broader linguistic category**, including idioms, collocations, phrasal verbs, etc.
- Named Entities are a **semantic subset** focused on identifying unique real-world referents.
- The intersection lies in **named entity MWEs** like "The United Nations" or "Silicon Valley", but this is just a subset of both classes.

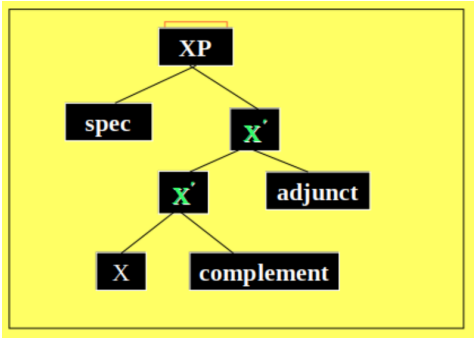
Conclusion:

- Some NEs are MWEs, but not all.
- Most MWEs are not NEs.
- While **NER systems often detect MWEs** (when they are entities), MWE detection and NER are **distinct tasks** with different objectives.

Question-3

a. **Define complements and adjuncts with reference to X-bar theory. How do they differ in terms of syntactic structure and semantic contribution? [2.5]**

b. **Using a sentence from an Indian language, identify a complement and an adjucnt. Explain how these distinctions affect parsing strategies in constituency vs. dependency frameworks. [2.5]**



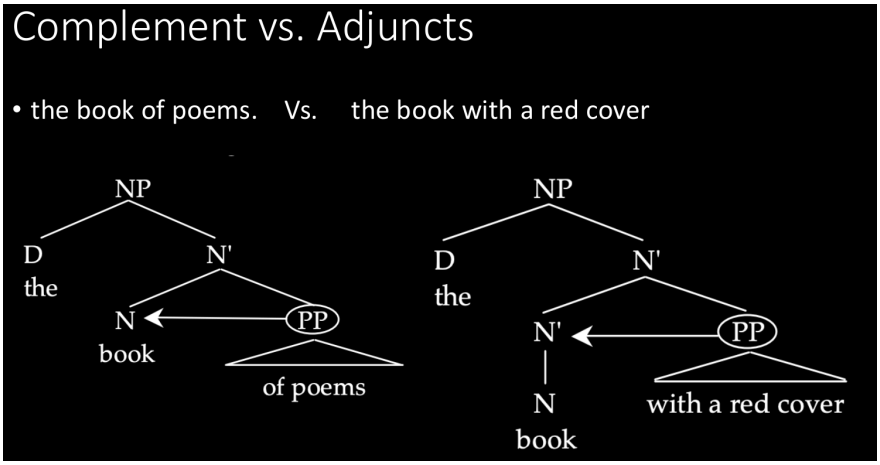
1. Define complements and adjuncts with reference to X-bar theory. How do they differ in terms of syntactic structure and semantic contribution? [2.5 marks]

Definition:

In X-bar theory, syntactic structures are built hierarchically using three levels: the head (X), intermediate projection (X′), and maximal projection (XP). Within this framework:

Adjunct: An XP that is a sister to a single bar level (N', V', A', or P') and a daughter of a single bar level (N', V', A', or P').

Complement: An XP that is a sister to a head (N, V, A, P), and a daughter of a single bar level (N', V', A', or P').



Syntactic Structure Differences:

- Complement is closer to the head in the tree structure (e.g., [X′ [X head] [Complement]]).
- Adjunct is structurally more external and recursive (e.g., [X′ [X′ …] [Adjunct]]).

Semantic Contribution Differences:

- Complements are semantically selected by the head; their presence affects grammaticality.
- Adjuncts are optional and add additional, often circumstantial information (time, location, manner, etc.) without changing core meaning or selectional requirements.

> **Marking Key (2.5 marks):**

- 1.5 mark: Correct definitions of complement and adjunct in X-bar theory
- 0.5 mark: Clear explanation of structural difference
- 0.5 mark: Clear explanation of semantic difference

---

2. Using a sentence from an Indian language, identify a complement and an adjunct. Explain how these distinctions affect parsing strategies in constituency vs. dependency frameworks. [2.5 marks]

Example sentence in Hindi:

> राम ने सीता को किताब दी।
>
> (Rām ne Sītā ko kitāb dī.)
>
> *"Ram gave a book to Sita."*

- Complement: *किताब (kitāb) – "book"* → required by the verb दी (dī) (gave)
- Adjunct: *राम ने (Rām ne) – "Ram (ERG)"* → provides agent information, optional in many contexts depending on focus

> Alternatively, in a better example:
>
> राम ने सीता को पार्क में किताब दी।
>
> *"Ram gave a book to Sita in the park."*

- Complement: *सीता को (Sītā ko) – "to Sita"*
- Adjunct: *पार्क में (pārk meṅ) – "in the park"*

Parsing Differences:

- Constituency parsing (Phrase Structure Grammar):
  - Emphasizes hierarchical phrase groupings.
  - Complements are part of the VP (Verb Phrase) and directly linked to the verb.
  - Adjuncts are added higher up and may attach ambiguously, requiring disambiguation strategies.
  - Parsing must decide between multiple possible attachments (e.g., adjunct attachment ambiguity).
- Dependency parsing:
  - Focuses on direct head–dependent relations.
  - Complements are core dependents of the verb, often with fixed dependency labels (e.g., `obj`, `iobj`).
  - Adjuncts are non-core dependents (e.g., `obl`, `advmod`), often optional.
  - Parsing is simpler in some ways as there's no need to build constituent hierarchies, but labeling adjuncts correctly still requires semantic insight.

> **Marking Key (2.5 marks):**

- 1.0 mark: Indian language sentence with correct identification of a complement
- 0.5 mark: Correct identification of an adjunct
- 0.5 mark: Correct explanation of difference in constituency parsing
- 0.5 mark: Correct explanation of difference in dependency parsing

---

Question-4

a. **Compare constituency parsing and dependency parsing in terms of their representation of syntactic structure. Which is more suitable for morphologically rich and free word order langauges, and why? [2.5]**

Constituency Parsing:

- Represents sentence structure using phrase structure trees based on constituents (e.g., NP, VP).
- Emphasizes hierarchical groupings of words into nested phrases.
- Example (English):

  [S [NP The boy] [VP kicked [NP the ball]]]

Dependency Parsing:

- Represents syntactic structure through binary relations (dependencies) between individual words.

- Focuses on head-dependent relations rather than phrasal groupings.
- Example (English):

  kicked → boy (nsubj), *kicked → ball (obj)/.*

Suitability for Morphologically Rich & Free Word Order Languages:

- Dependency parsing is more suitable for languages like Hindi, Tamil, etc.
- These languages have rich morphology (e.g., case markers, verb inflections) and flexible word order, making phrase structure less consistent.
- Dependency structures are less sensitive to word order and can rely on morphological cues to determine grammatical relations.

Marking Scheme (2.5 marks):

- 1.0: Definition of Constituency Parsing, Dependency Parsing
- 0.5: Suitability discussion → DP more suitable
- 1.0: Justification for suitability (morphology + word order)

b. **Explain how morphological information (e.g., case markers, verb agreements) can assist in dependency parsing. Use examples from any indian langauge. [2.5]**

Morphological features like case markers and verb agreement help identify syntactic roles (subject, object, etc.) and improve parsing accuracy in Indian languages.

Example in Hindi:

Sentence:

लड़के ने गेंद को मारा।

(*Laḍke ne geṃd ko mārā.*)

Translation: “The boy hit the ball.”

- ने (ne) → ergative marker, signals that लड़के (boy) is the subject (agent).
- को (ko) → accusative/dative marker, indicates गेंद (ball) is the object (patient).
- Verb मारा (hit) agrees in gender/number with the object in ergative constructions.

These markers guide the parser in establishing:

- "मारा" → "लड़के" (subject relation)
- "मारा" → "गेंद" (object relation)

Even with free word order like:

गेंद को लड़के ने मारा।

Parsing is still accurate due to the morphological markers.

Marking Scheme (2.5 marks): (not strictly followed)

- 0.5: General role of morphology in parsing
- 0.5: Case marker usage
- 0.5: Verb agreement usage
- 0.5: sentence example
- 0.5: Explanation of how parsing is aided despite word order changes

---

Question-5

a. **Explain the conceptual foundations of paninian grammer and how karaka theory is applied in parsing. [2]**

Paninian Grammar, rooted in the ancient Sanskrit grammatical tradition (notably in Pāṇini's *Ashtadhyayi*), emphasizes a dependency-based syntactic structure. Words are connected via binary relations, typically verbs governing nouns, rather than constituent phrase structures (like in Western grammars). The focus is on semantic-syntactic roles rather than surface word order.

A central idea is the Karaka Theory, which maps semantic roles to syntactic positions. Each Karaka indicates a relationship between the action (verb) and participants (nouns), such as:

- Karta (Agent): Doer of the action
- Karma (Patient): Object of the action
- Karana: Instrument
- Sampradana: Beneficiary or recipient
- Apadana: Source or point of separation
- Adhikarana: Location or context

These Karakas help in determining the roles independent of word order, thus making Paninian grammar especially suitable for Indian languages, which often exhibit free word order.

Marking Scheme [2 marks]:

- 1 mark for explanation of Paninian dependency grammar
- 1 mark for explaining Karaka roles and their function in parsing

b. **Take an example sentence from any indian language and show both paninian and UD annotations. Comment on their difference in handling semantic roles and word order. [3]**

Example sentence (Hindi):

राम ने सीता को फल दिया।

(Rām ne Sītā ko phal diyā.)

Translation: Ram gave fruit to Sita.

(a) Paninian Annotation:

Word	Karaka Role	Relation
राम ने	Karta	Agent
सीता को	Sampradana	Recipient
फल	Karma	Theme/Object
दिया	Verb	Action

<Dependency tree (Paninian style)>

(b) Universal Dependencies (UD):

Word	UD Relation	Head
राम	nsubj	दिया
ने	case	राम
सीता	iobj	दिया
को	case	सीता
फल	obj	दिया

दिया	root	-

<UD tree>

(c) Commentary on Differences:

- Semantic Roles: Paninian grammar explicitly encodes semantic roles (Karta, Karma, etc.), while UD uses syntactic relations (nsubj, obj, iobj), which are less directly tied to meaning.
- Word Order Sensitivity: Paninian annotation is independent of word order, focusing on vibhakti markers (e.g., 'ने', 'को'). UD, while robust, is more sensitive to surface structure, requiring separate `case` annotations to capture postpositions.

Marking Scheme [3 marks]:

- 1 mark for correct Paninian annotation
- 1 mark for correct UD annotation
- 1 mark for insightful comparison of roles and word order handling

---

Question-6

a. **What are the major challenges in ASR for Indian Languages? Discuss how linguistic diversity (e.g., Phonetics, morphology, prosody) contributes to these challenges. [2.5]**

   Marking Scheme (Any five relevant challenges to Indian Languages → 5 × 0.5)

   - 0.5: Code-switching and multilingualism
   - 0.5: Phonetic diversity
   - 0.5: Morphological complexity
   - 0.5: Dialects and accents
   - 0.5: Resource scarcity

b. **Explain the fundamental differences between ASR and TTS in terms of input, output, and core processing stages. [2.5]**

Feature	**ASR (Automatic Speech Recognition)**	**TTS (Text-to-Speech)**
Input	Speech waveform	Text (graphemes or phonemes)
Output	Text transcript	Speech waveform
Core Stages	*(0.5 marks each)*	*(0.5 marks each)*
- **Feature Extraction**	Extracts acoustic features (e.g., MFCC) from speech	Converts text to linguistic features (phonemes, stress)
- **Modeling**	Acoustic model + Language model + Decoder	Duration model + Acoustic model + Vocoder
- **Inference**	Maps audio to the most probable text sequence	Generates speech waveform from linguistic input

Marking Scheme

- 0.5: Input comparison
- 0.5: Output comparison
- 1.5 (0.5 × 3): Core processing stages (feature extraction, modeling, inference)

Question-7

a. **Many informal texts contain abbreviatons and ellipses (e.g., "U.S", "Dr.", "etc.", "Wait... What?!"). These make sentence splitting with regular expressions difficult. Design a regex-based strategy to split sentences while avoiding splits inside abbreviations and ellipses. Explain how your regex handles such exceptions. [2.5]**

- `(?<!\b...)` : Negative lookbehind prevents splits after known **abbreviations**.
- `(?<!\.\.)` : Avoid splitting after multiple periods (`..`, `...`, `....`) — handles **ellipses**.
- `(?<=[.?!])` : Ensures punctuation is the real delimiter.
- `(?=[A-Z])` : Ensures next word starts with capital (probable sentence start).

Marking Scheme (2.5 Marks):

Criterion	Marks
Handles common abbreviations without splitting	1.0
Handles ellipses correctly	0.5
Splits correctly at `.` `!` `?` only when followed by capital letter	0.5
Explanation of how regex handles cases	0.5

b. **Write a single regular expression that matches the following in social media text: [2.5]**

Mentions

hastags

Emojis from the Unicode range U+1F600 to U+1F64F (emoticons...) which may appear consequetively_

Regex:

```
(?:@\w+)|(?:#\w+)|(?:[\U0001F600-\U0001F64F]+)
```

Explanation:

- `@\w+` matches **mentions** like `@user123`
- `#\w+` matches **hashtags** like `#awesome_day`
- `[\U0001F600-\U0001F64F]+` matches one or more **emojis** in the Unicode emoticon range (consecutive ones too)

Marking Scheme (2.5 Marks):

Criterion	Marks
Correct mention regex (`@\w+`)	0.5
Correct hashtag regex (`#\w+`)	0.5
Correct emoji Unicode range usage	1.0
Handles **consecutive emojis** with `+`	0.5

Question-8

a. **Both HMMs and CRFs are used for sequence labeling tasks. What is the key difference in how they model probability, and why does it matter? [2.5]**

Key Difference in How HMMs and CRFs Model Probability [2.5 Marks]

- **HMM (Hidden Markov Model)** is a **generative model**: it models the **joint probability** of the observations and the labels:

  P(X,Y)=P(Y)P(X ∣ Y)P(X, Y) = P(Y)P(X|Y)

This means HMMs must model how observations (words) are generated from labels (tags).

- **CRF (Conditional Random Field)** is a **discriminative model**: it directly models the **conditional probability** of labels given observations:

  P(Y ∣ X)P(Y|X)

  CRFs focus only on the probability of labels given the data, not how the data itself is generated.

- **Why it matters:**

  CRFs can **relax independence assumptions** about the input (observations) and **include rich, overlapping features** of the input (e.g., word shapes, prefixes, surrounding words), which HMMs cannot do due to their generative nature. This makes CRFs generally more **flexible and accurate** for real-world NLP tasks.

Marking Scheme (Total: 2.5 marks):

- 1.0: Clearly states HMM is generative, CRF is discriminative.
- 0.5: Correct formulation of probability modeling.
- 0.5: Highlights independence assumption differences.
- 0.5: Explains practical consequence (e.g., CRFs handle richer features).

b. **Can HMMs and CRFs handle the same kinds of linguistic dependencies? Discuss with an example why one might perform better than the other in tasks like POS tagging or NER. [2.5]**

Handling Linguistic Dependencies & Comparative Performance [2.5 Marks]

- **HMMs** assume that the current state (tag) only depends on the **previous state**, and the observation only depends on the **current state**. This **Markov assumption** limits modeling long-range or complex dependencies.
- **CRFs**, in contrast, can incorporate **arbitrary features** of the entire input sequence, including **non-local context** (e.g., surrounding words, capitalization, affixes, etc.), making them better at capturing complex dependencies.
- **Example - Named Entity Recognition (NER):**

  Consider recognizing names like *"New York City"*. An HMM might struggle due to its limited feature representation and strong independence assumptions. A CRF can include features like:

  - Word shape ("capitalized"),
  - Previous/following words,
  - Prefixes/suffixes (e.g., "City" ends the phrase).

  These help it **better model multi-token entities** or contexts, resulting in higher accuracy in tasks like **NER** or **POS tagging**.

Marking Scheme (Total: 2.5 marks):

- 1.0: Identifies HMM's limited context and independence assumptions.
- 0.5: Describes CRF's ability to handle rich features.
- 0.5: Gives a relevant example (e.g., NER or POS tagging).
- 0.5: Explains why CRF performs better in the example.

Section-B: Data Annotation (30 Marks)

Sentences:

1. The young athlete with remarkable stamina won three medals at the national meet.
2. The curious boy from hyderabad who always asked questions finally got selected for the science fair.
3. During the Republic Day parade in Delhi, Air Chief Marshal Rakesh Kumar saluted the President with pride.
4. After weeks of back and forth, the startup's pitch deck woved the angel investors at the Bengaluru conclave.

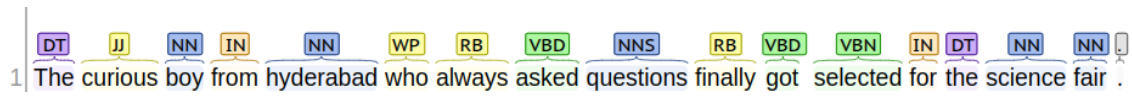
Questions:

9. Perform morphological analysis, POS tagging, and Chunking for sentence (a) and (b).
10. Identify Multi-Word Expressions (MWEs) in sentences (c) and (d), and annotate the type.
11. Identify and annotate Named Entities (person, organisation etc.) in sentences (c) and (d).
12. Draw X-bar schema for sentences (a) and (b), and clearly mark complements vs. adjuncts.
13. Draw a Dependency Tree with syntactic relations (either Panini based / UD based) for sentences (a) and (b).

Question 9: Morphological Analysis, POS Tagging, and Chunking for sentences (a) and (b)

(a) Sentence:

The curious boy from Hyderabad who always asked questions finally got selected for the science fair.

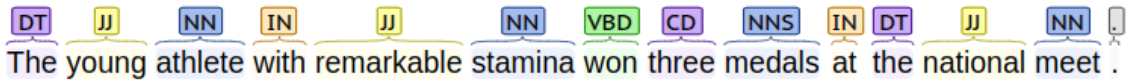


Morphological Analysis | POS | Chunking:

```
[The/DT] [curious/JJ] [boy/NN]     → NP
[from/IN] [Hyderabad/NNP]          → PP
[who/WP] [always/RB] [asked/VBD] [questions/NNS]  → SBAR/Clause
[finally/RB] [got/VBD] [selected/VBN]          → VP
[for/IN] [the/DT] [science/NN] [fair/NN]       → PP
```

(b) Sentence:

The young athlete with remarkable stamina won three medals at the national meet.



Morphological Analysis | POS | Chunking:

```
[The/DT] [young/JJ] [athlete/NN]              → NP
[with/IN] [remarkable/JJ] [stamina/NN]        → PP (modifier)
[won/VBD]                                     → VP
[three/CD] [medals/NNS]                       → NP
[at/IN] [the/DT] [national/JJ] [meet/NN]      → PP
```

	Marks
Correct POS tags for all words (1 mark each for sentence a & b)	2
Accurate morphological analysis (base + affixes, tense, number, etc.)	2
Correct chunking (identifying NPs, VPs, PPs, etc.)	2

Question 10: MWEs (Multi-Word Expressions) in (c) and (d)

(c)

During the Republic Day parade in Delhi, Air Chief Marshal Rakesh Kumar saluted the President with pride.

- **"Republic Day"** – *Named entity (event), fixed expression*
- **"Air Chief Marshal"** – *Institutional title, compositional and conventionalized MWE*
- **"with pride"** – *Adverbial collocation (light idiomatic MWE)*

(d)

After weeks of back and forth, the startup's pitch deck wowed the angel investors at the Bengaluru conclave.

- **"back and forth"** – *Idiomatic expression (frozen MWE)*
- **"pitch deck"** – *Domain-specific collocation (semi-fixed MWE), compound*
- **"angel investors"** – *Financial domain collocation (conventionalized MWE)*
- **"Bengaluru conclave"** – *Named entity + noun (event-type MWE)*

Question 11: Named Entity Recognition in (c) and (d)

(c)

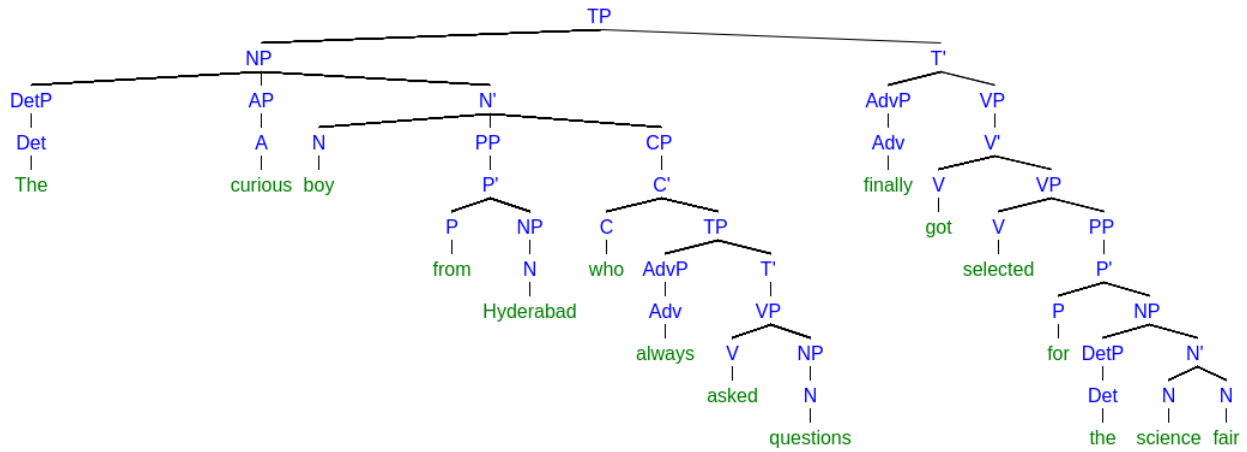
- **Republic Day** – *Event*
- **Delhi** – *Location*
- **Air Chief Marshal Rakesh Kumar** – *Person (with Title)*
- **President** – *Title (possible Person depending on context)*

(d)

- **Bengaluru** – *Location*
- **angel investors** – *Organization/Group (can be Person based on interpretation)*
- **startup** – *Organization (implied)*

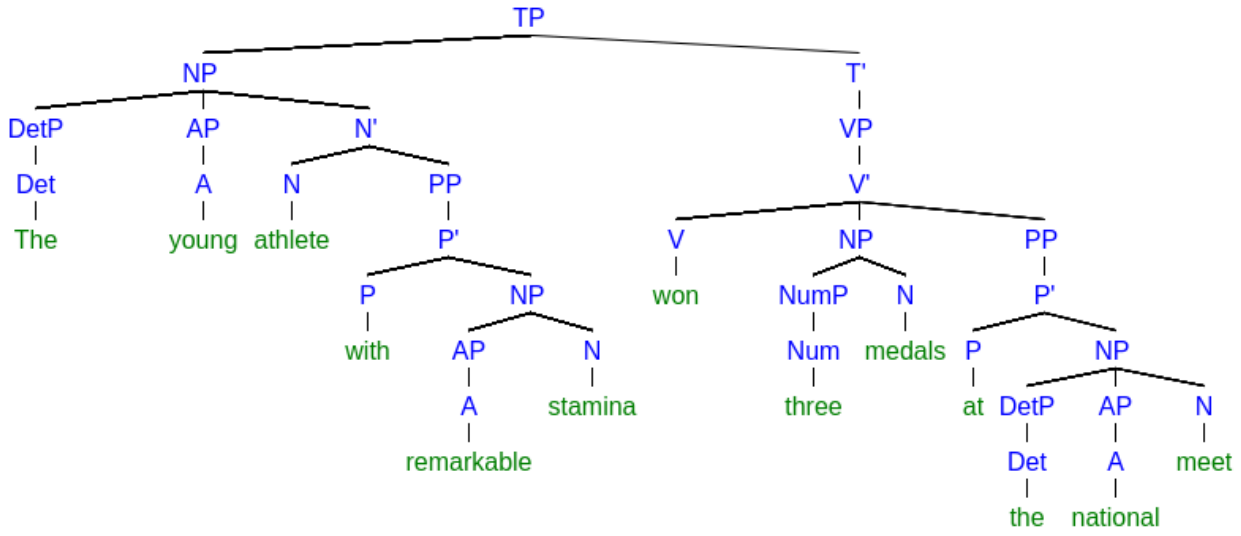
Question 12: X-bar Schema for (a) and (b)

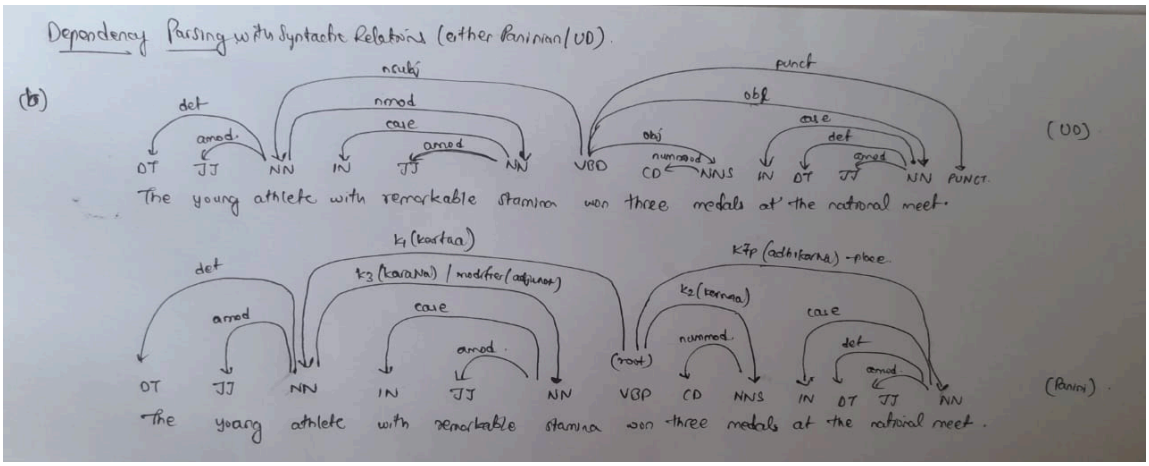
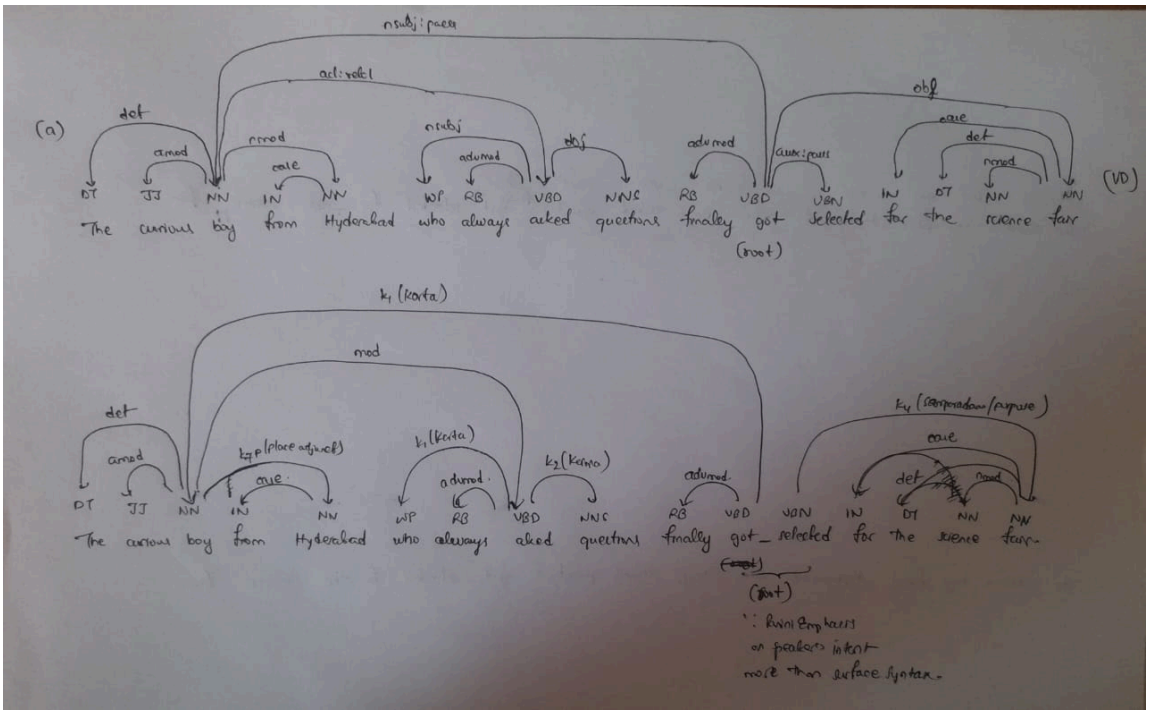
(a) Approximate tree (note that some corrections are manually checked)



Phrase/Constituent	Type	Justification
"curious"	Adjunct	Optional adjective modifying "boy"
"from Hyderabad"	Adjunct	Optional PP modifying "boy"
"who always asked questions"	Adjunct	Relative clause modifying "boy"
"finally"	Adjunct	Optional adverb modifying the VP
"for the science fair"	**Complement**	Required to complete meaning of "got selected"

(b) Approximate tree (note that some corrections are manually checked)



Phrase/Constituent	Type	Justification
"young"	Adjunct	Optional adjective modifying "athlete"
"with remarkable stamina"	Adjunct	Optional PP modifying "athlete"
"three medals"	**Complement**	Required object of the verb "won"
"at the national meet"	Adjunct	Optional PP giving location

Question 13: Dependency Tree (Universal Dependencies) for (a) and (b)